Swerve Drive with YAGSL!

(Yet Another Generic Swerve Library)

By Ethan Spickard of Team 3939

Link to the Docs: https://docs.yagsl.com/

What is YAGSL?

- **YAGSL** is a library for WPILib that does most of your swerve code for you!
  - *Steal from the best, invent the rest. -Mike Corsetto, Team 1678*
- It can provide your robot a code base set up for:
  - Robot odometry off visioning systems, like **Limelight** or **PhotonVision**.
  - Autonomous construction with **PathPlanner**.
  - Automatically aiming to specific points in the field.
  - Telemetry for Advantage Scope and various dashboards.
  - Simulation capabilities.
  - Much much more as it continues to be developed!

Steps to get started:

1. Create a robot configuration file.
    1. Record the necessary specs of your robot.
    2. Use the online config generator to create a robot configuration file.

2. Set up the Swerve Drive environment in VS Code.
    1. Install the YAGSL library in WPILib.
    2. Copy in the example SwerveSubsystem/VisionSubsystem/ RobotContainer from the YAGSL website.
    3. Copy in the generated configuration folder.

Recording the specs of the robot.

- YAGSL has no idea what your robot is using until you tell it!
- Record the following things:
  - Gyro Model
  - Robot Weight
  - Wheel Grip Coefficient
  - Wheel Diameter
  - *Absolute Encoder Offsets**
  - *PIDF Tuning**
  - Motor Types
  - Motor CAN IDs
  - Module Gear Ratios
  - Position of Modules Relative to Robot Center
- The most tedious part of the process, but accurate measurements will lead to high robot accuracy and precision.
- **Here is a worksheet that will work you through the process!**

***Must be found experimentally!**

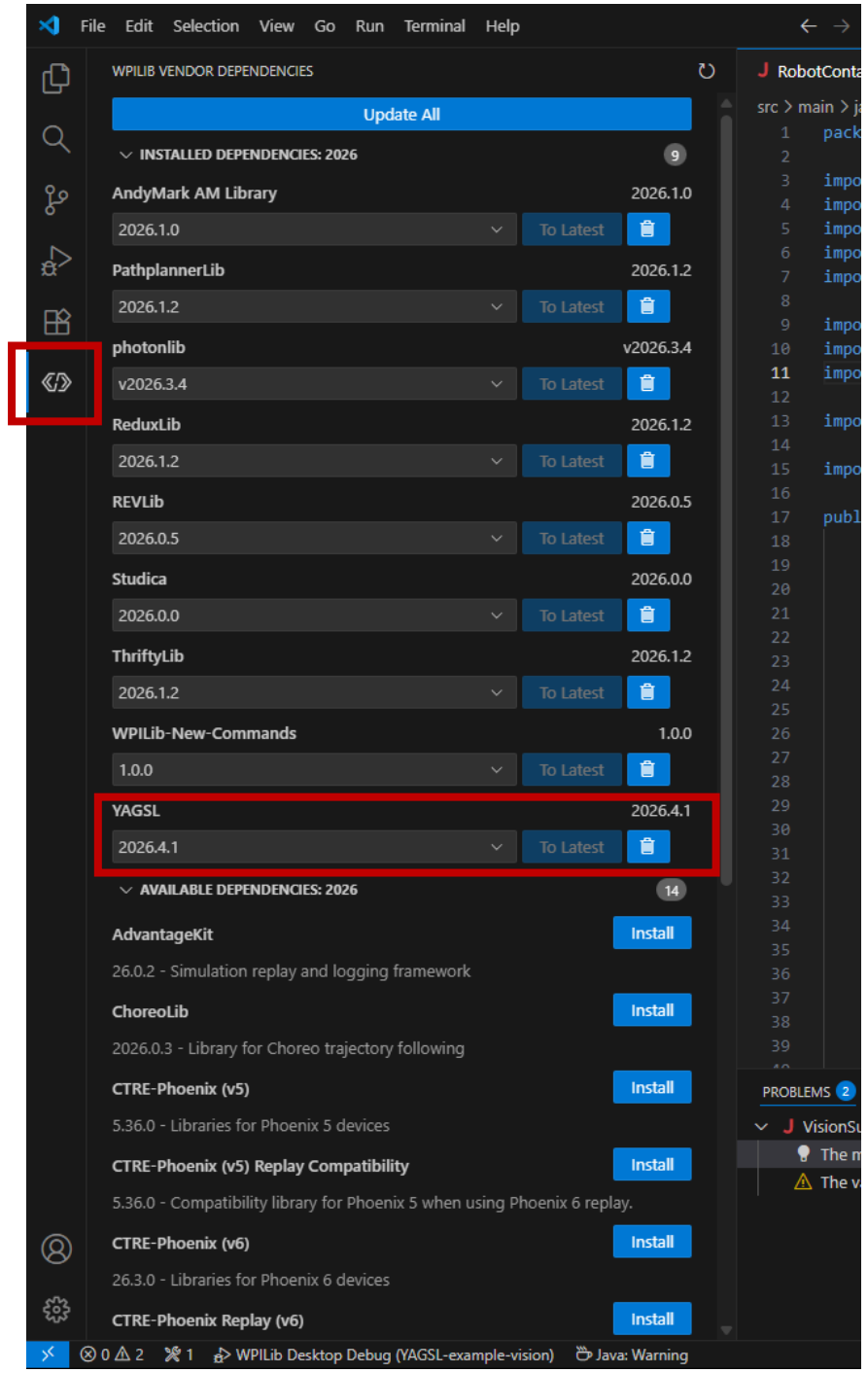
Create robot configuration files.

- **Use the generator here!**
- The information we gathered previously will be converted into the format that the robot can use to know exactly how our robot should behave.
- We'll save the generated folder in the downloaded zip file named swerve for later once we have our VS Code setup.

swerve

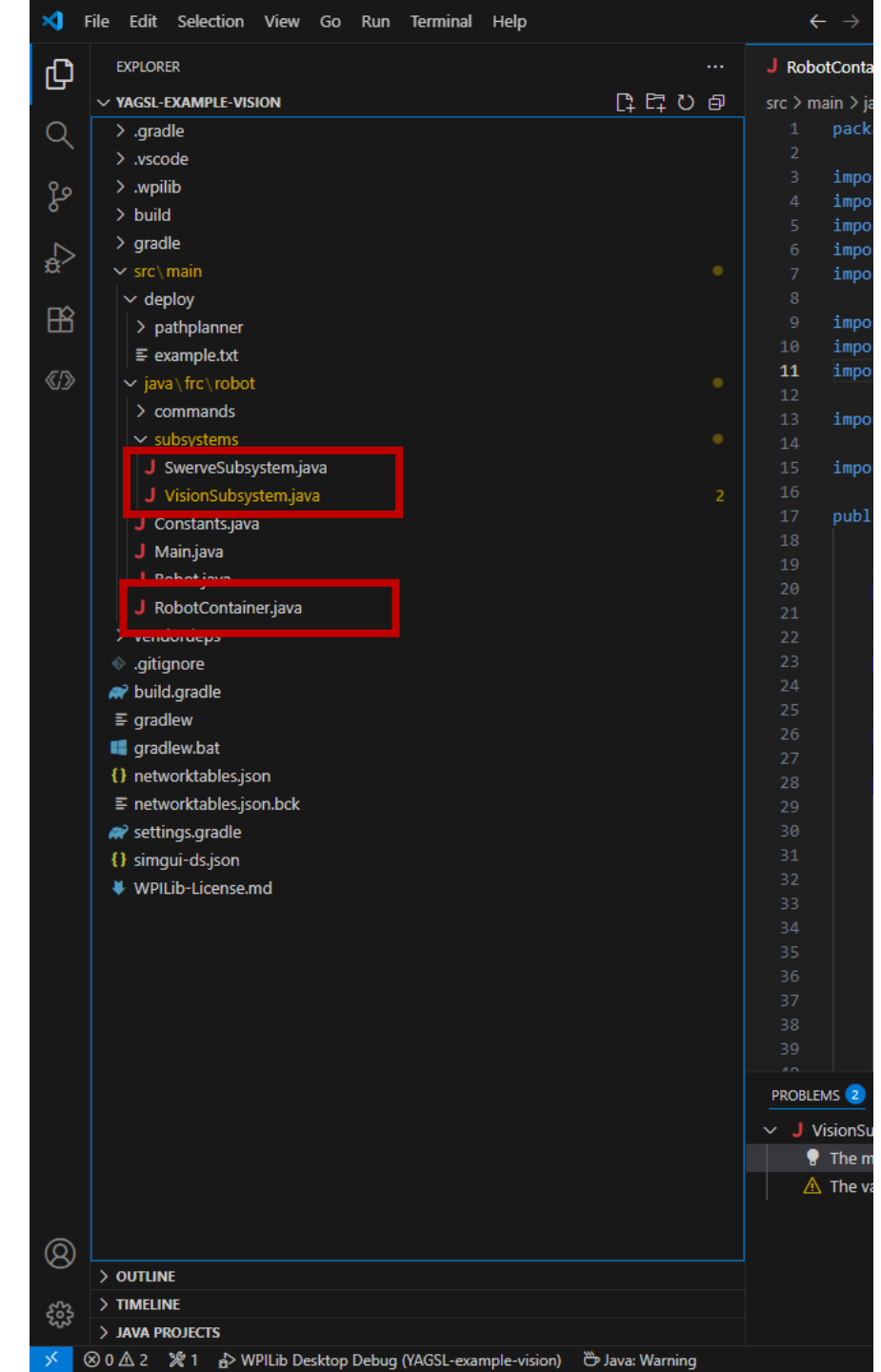
Installing the appropriate libraries.

- WPILib doesn't know that we want to use YAGSL yet, so we need to tell it to include it in our codebase.
- Make sure to also import the libraries of the motors you are using.



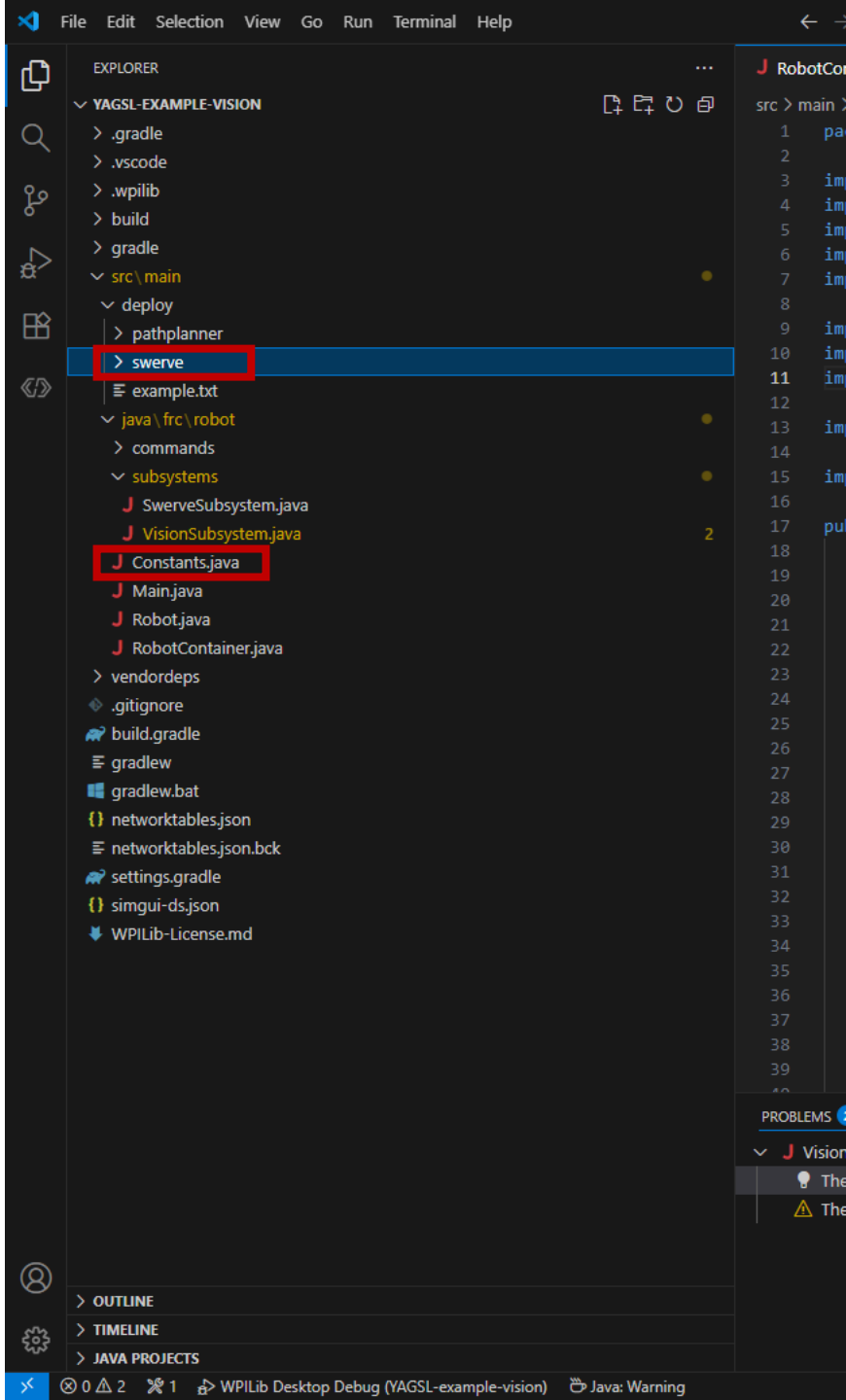
Copy the example SwerveSubsystem/ VisionSubsytem/ RobotContainer files.

- A version is available on the **YAGSL Website** that doesn't use visioning, but the one included with the presentation includes a vision subsystem.
- You should completely copy the SwerveSubsytem, but if you know what you're doing, you can only copy the bits you need from the example RobotContainer.



Copy in the generated robot configuration folder.

- Extract the .zip file downloaded from the configuration generator and place its contents in the deploy directory.
- If your robot's specs change at any point (weight, motors, etc.) go through the online generator and download it again to make changes instead of trying to mess with the .json files.
- At this point, also put your robots physical max speed in Constants.java.



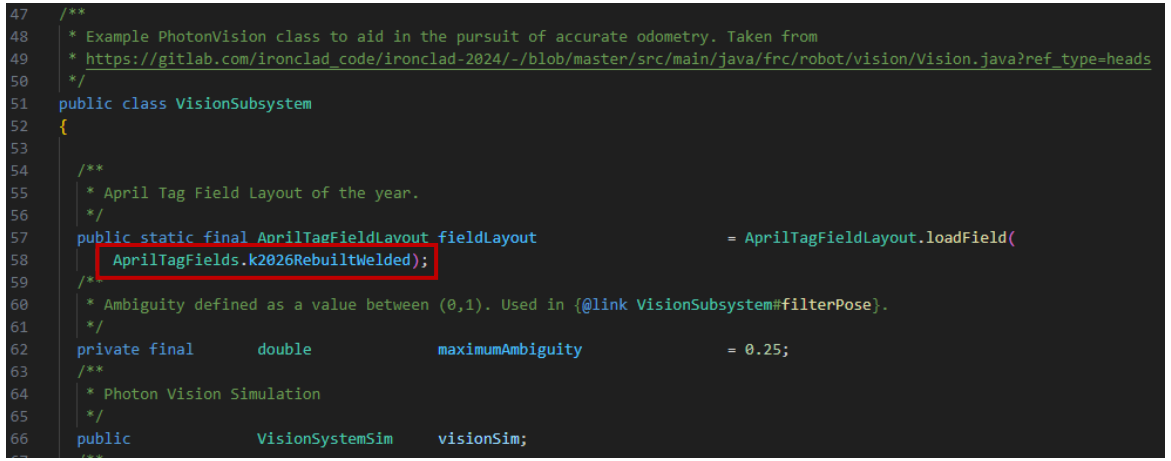
Great, but what does this code even do?

In short, **a lot**. The two subsystems we imported contain **over 1300 lines of code.**

We'll go over some useful information on what is important to know.

VisionSubsystem

- Not much for us to do here, this code is mostly **PhotonVision** math that gets fed straight into SwerveSubsystem.
- Every year, change the **AprilTag** layout.
- Add the specs of your camera into **enum Cameras**, the cameras currently in the code are examples.



SwerveSubsystem

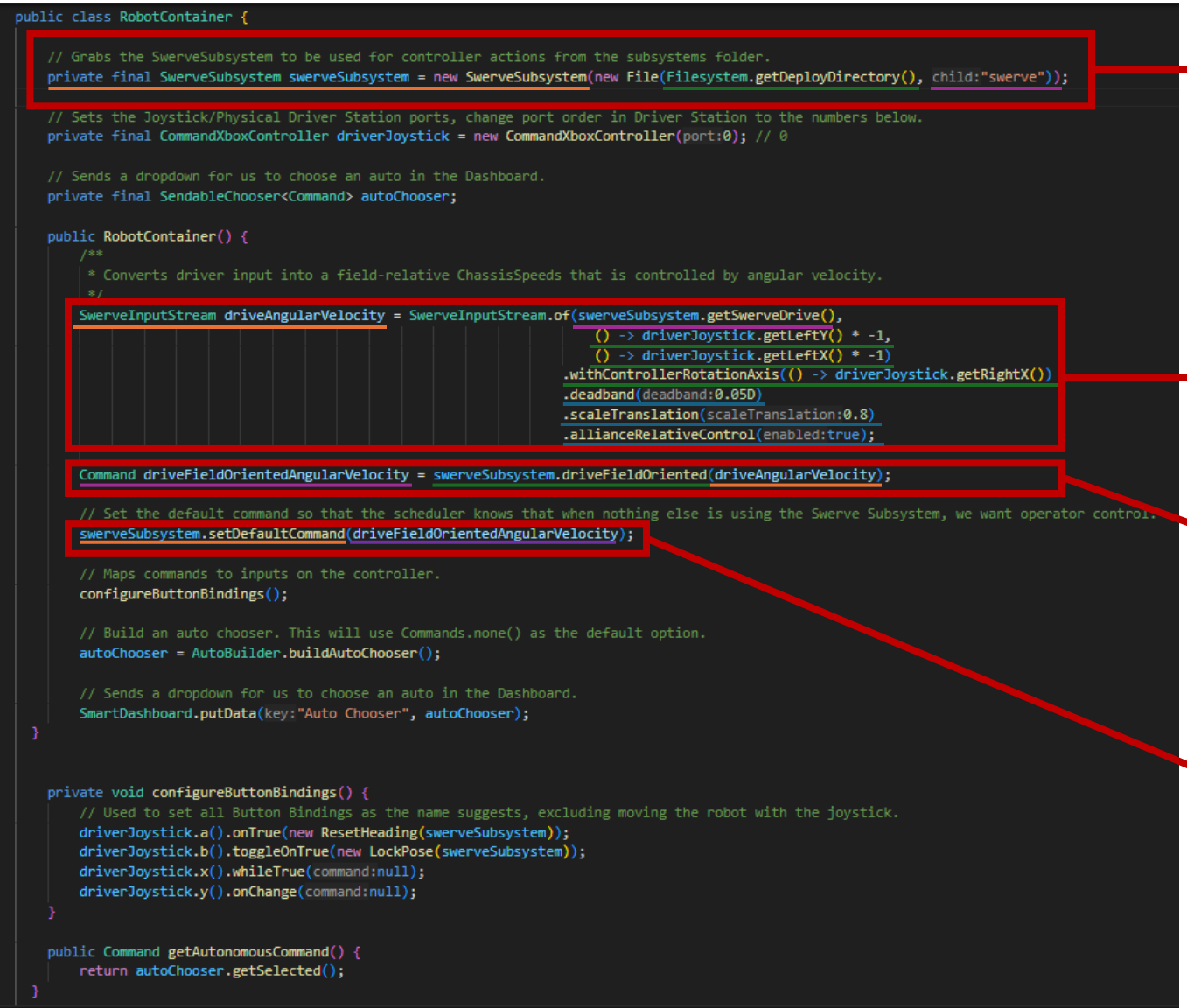
Not much code for us to change either, but there are some useful methods that we can use in our commands.

The starting pose the robot will think it's at if it doesn't get any vision readings or doesn't start from an auto.

```
80  {
81    boolean blueAlliance = DriverStation.getAlliance().isPresent() && DriverStation.getAlliance().get() == Alliance.Blue;
82    Pose2d startingPose = blueAlliance ? new Pose2d(new Translation2d(Meter.of(magnitude:1),
83                                                                      Meter.of(magnitude:4)),
84                                                    Rotation2d.fromDegrees(degrees:0))
85                                       : new Pose2d(new Translation2d(Meter.of(magnitude:16),
86                                                                      Meter.of(magnitude:4)),
87                                                    Rotation2d.fromDegrees(degrees:180));
88    // Configure the Telemetry before creating the SwerveDrive to avoid unnecessary objects being created.
89    SwerveDriveTelemetry.verbosity = TelemetryVerbosity.HIGH;
90    try
91    {
92      swerveDrive = new SwerveParser(directory).createSwerveDrive(Constants.DriveConstants.kPhysicalMaxSpeedMetersPerSecond, startingPose);
93      // Alternative method if you don't want to supply the conversion factor via JSON files.
94      // swerveDrive = new SwerveParser(directory).createSwerveDrive(maximumSpeed, angleConversionFactor, driveConversionFactor);
95    } catch (Exception e)
96    {
97      throw new RuntimeException(e);
98    }
99    swerveDrive.setHeadingCorrection(state:false); // Heading correction should only be used while controlling the robot via angle.
100   swerveDrive.setCosineCompensator(enabled:false);//!SwerveDriveTelemetry.isSimulation); // Disables cosine compensation for simulations since it ca
101   swerveDrive.setAngularVelocityCompensation(useInTeleop:true,
102                                              useInAuto:true,
103                                              angularVelocityCoeff:0.1); //Correct for skew that gets worse as angular velocity increases. Start with
104   swerveDrive.setModuleEncoderAutoSynchronize(enabled:false,
105                                               deadband:1); // Enable if you want to resynchronize your absolute encoders and motor encoders periodic
106   // swerveDrive.pushOffsetsToEncoders(); // Set the absolute encoder to be used over the internal encoder and push the offsets onto it. Throws warn
```

Some potentially useful settings, but can likely be left alone if you aren't sure if you should change them.

RobotContainer



```java
public class RobotContainer {

    // Grabs the SwerveSubsystem to be used for controller actions from the subsystems folder.
    private final SwerveSubsystem swerveSubsystem = new SwerveSubsystem(new File(Filesystem.getDeployDirectory(), child:"swerve"));

    // Sets the Joystick/Physical Driver Station ports, change port order in Driver Station to the numbers below.
    private final CommandXboxController driverJoystick = new CommandXboxController(port:0); // 0

    // Sends a dropdown for us to choose an auto in the Dashboard.
    private final SendableChooser<Command> autoChooser;

    public RobotContainer() {
        /**
         * Converts driver input into a field-relative ChassisSpeeds that is controlled by angular velocity.
         */
        SwerveInputStream driveAngularVelocity = SwerveInputStream.of(swerveSubsystem.getSwerveDrive(),
                                                                      () -> driverJoystick.getLeftY() * -1,
                                                                      () -> driverJoystick.getLeftX() * -1)
                                                                  .withControllerRotationAxis(() -> driverJoystick.getRightX())
                                                                  .deadband(deadband:0.05D)
                                                                  .scaleTranslation(scaleTranslation:0.8)
                                                                  .allianceRelativeControl(enabled:true);

        Command driveFieldOrientedAngularVelocity = swerveSubsystem.driveFieldOriented(driveAngularVelocity);

        // Set the default command so that the scheduler knows that when nothing else is using the Swerve Subsystem, we want operator control.
        swerveSubsystem.setDefaultCommand(driveFieldOrientedAngularVelocity);

        // Maps commands to inputs on the controller.
        configureButtonBindings();

        // Build an auto chooser. This will use Commands.none() as the default option.
        autoChooser = AutoBuilder.buildAutoChooser();

        // Sends a dropdown for us to choose an auto in the Dashboard.
        SmartDashboard.putData(key:"Auto Chooser", autoChooser);
    }


    private void configureButtonBindings() {
        // Used to set all Button Bindings as the name suggests, excluding moving the robot with the joystick.
        driverJoystick.a().onTrue(new ResetHeading(swerveSubsystem));
        driverJoystick.b().toggleOnTrue(new LockPose(swerveSubsystem));
        driverJoystick.x().whileTrue(command:null);
        driverJoystick.y().onChange(command:null);
    }

    public Command getAutonomousCommand() {
        return autoChooser.getSelected();
    }
}
```

"Imports" the SwerveSubsystem file and tells it to grab the folder swerve from the deploy directory.

Create an object where the SwerveSubsystem can be controlled by the game controller with some extra settings.

Use the previously created object and save a version that is field centric.

When nothing else is using the SwerveSubsystem, set the current command to operator-controlled field centric drive.

And finally, some neat things that you can do with YAGSL.

Locking the Robot Pose.

- Moves your robot's wheels into an X formation so that is harder to be moved when stationary.
- Written as a Command class, all it does is call the existing SwerveSubsystem lock() function.

```
56    private void configureButtonBindings() {
57        // Used to set all Button Bindings as the name suggests, excluding moving the robot with the joystick.
58        driverJoystick.a().onTrue(new ResetHeading(swerveSubsystem));
59        driverJoystick.b().toggleOnTrue(new LockPose(swerveSubsystem));
60        driverJoystick.x().whileTrue(command:null);
61        driverJoystick.y().onChange(command:null);
62    }
```

```
1   // Resets the current rotation of the gyro to be considered 0.
2
3   package frc.robot.commands;
4
5   import edu.wpi.first.wpilibj2.command.Command;
6   import frc.robot.subsystems.SwerveSubsystem;
7
8   public class LockPose extends Command {
9     /** Creates a new ZeroHeading. */
10    private final SwerveSubsystem swerveSubsystem;
11    public LockPose(SwerveSubsystem swerveSubsystem) {
12      this.swerveSubsystem = swerveSubsystem;
13      addRequirements(swerveSubsystem);
14    }
15
16    // Called when the command is initially scheduled.
17    @Override
18    public void initialize() {
19      swerveSubsystem.lock();
20    }
21
22    // Called every time the scheduler runs while the command is scheduled.
23    @Override
24    public void execute() {
25      swerveSubsystem.lock();
26    }
27
28    // Called once the command ends or is interrupted.
29    @Override
30    public void end(boolean interrupted) {}
31
32    // Returns true when the command should end.
33    @Override
34    public boolean isFinished() {
35      return true;
36    }
37  }
38  //(^. .^)
```

Aiming the Robot towards a Point.

```
46      public RobotContainer() {
47          /**
48           * Converts driver input into a field-relative ChassisSpeeds that is controlled by angular velocity.
49           */
50          // Change the -1's to 1's to invert joystick directions!
51          SwerveInputStream driveAngularVelocity = SwerveInputStream.of(swerveSubsystem.getSwerveDrive(),
52                                                                        () -> driverJoystick.getLeftY() * -1,
53                                                                        () -> driverJoystick.getLeftX() * -1)
54                                                                    .withControllerRotationAxis(() -> driverJoystick.getRightX())
55                                                                    .deadband(deadband:0.05D)
56                                                                    .scaleTranslation(scaleTranslation:0.8)
57                                                                    .allianceRelativeControl(enabled:true);
58
59          Command driveFieldOrientedAngularVelocity = swerveSubsystem.driveFieldOriented(driveAngularVelocity);
60          swerveSubsystem.setDefaultCommand(driveFieldOrientedAngularVelocity);
61
62
63          // The on field locations of the hub from the right side of the blue alliance.
64          Pose2d redHubLocation = new Pose2d(x:11.920,y:4.021,Rotation2d.kZero);
65          Pose2d blueHubLocation = new Pose2d(x:4.612,y:4.021,Rotation2d.kZero);
66
67          // Supplies the following hubAim with the location that the robot needs to be facing.
68          Supplier<Pose2d> hubSupplier = () -> {
69              // Picks the hub location based on what alliance you're on.
70              Pose2d robotAim = (DriverStation.getAlliance().orElse(Alliance.Red) == Alliance.Blue) ? blueHubLocation : redHubLocation;
71              // Returns a pose that we can aim to.
72              return robotAim;
73          };
74
75          // Takes our AngularVelocity drive path and overwrites the rotation component to always face towards the correct hub.
76          // Also flips it 180 degrees to account for the side of the robot the shooter is on.
77          SwerveInputStream hubAim = driveAngularVelocity.copy().aim(hubSupplier).aimHeadingOffset(Rotation2d.fromDegrees(degrees:180)).aimHeadingOffset(enabled:true).aimWhile(trigger:true);
78          // Gives us a command that we can bind to the controller.
79          driveAim = swerveSubsystem.driveFieldOriented(hubAim);
80
```

Driving the Robot to a Point.

```
82          // The locations of the spots to climb on the field.
83          Pose2d blueClimbLocation = new Pose2d(x:1.504,y:3.719,Rotation2d.kZero);
84          Pose2d redClimbLocation = new Pose2d(x:15.020,y:3.719,Rotation2d.kZero);
85
86          // Supplies the following driveToPose command with the location that the robot should go to.
87          Supplier<Pose2d> climbSupplier = () -> {
88              // Picks the hub location based on what alliance you're on.
89              Pose2d climbLocation = (DriverStation.getAlliance().orElse(Alliance.Red) == Alliance.Blue) ? blueClimbLocation : redClimbLocation;
90              return climbLocation;
91          };
92
93          // Creates a command that we can bind to the controller.
94          driveToPose = swerveSubsystem.driveToPose(climbSupplier.get());
```

-
-
-

```
107     private void configureButtonBindings() {
108         // Used to set all Button Bindings as the name suggests, excluding moving the robot with the joystick.
109         driverJoystick.a().onTrue(new ResetHeading(swerveSubsystem));
110         driverJoystick.b().toggleOnTrue(new LockPose(swerveSubsystem));
111         driverJoystick.x().whileTrue(driveToPose);
112         driverJoystick.y().onChange(command:null);
113     }
```

And More!

- As one of the most popular swerve drive libraries, if there's something you want to do, it's likely very easy to do with YAGSL.
- Overall, YAGSL is a solution that takes a (relatively) short time to set up and maintain your drivetrain with features you'll need.
- A big community exists on Chief Delphi/Discord, so you likely find answers to any problem you may have.

Thank you for your time!

Questions?

Link to the Docs: https://docs.yagsl.com/

3939 BOTETOURT 4-H